# Leaderboard UML Planning Document

Rules & Validation + Quality-Testing

What we are implementing:

For this submission, we are modelling the leaderboard only (so not the matchmaking)
The leaderboard diagram will feature three main aspects:

1. View Leaderboard,
2. View My Score,
3. Submit Score (update the leaderboard).

Leaderboard Actors:

For actors, we are thinking of 2 main actors (for now):

1. The Player (the primary actor for the leaderboard)
2. The Game Server (the secondary actor that will store and return the leaderboard data for the game)

Use Case Diagram Plan:

The name for the system boundary can be straightforward: Leaderboard
Use cases we can/will include within it are:

1. View Leaderboard (shows the top N players)
2. View My Score (shows the users highscore | or shows the user's scores (a numbered list of best to worst, potentially)
3. Submit Score (this is just adding a new score/scores onto the leaderboard)

Assumptions for the Diagram:

I think it's fair to say that we assume:

1. Scores are stored server-side and accessed via network requests (similar to how the platform core's game architecture planning works).
2. Each specific score belongs to a specific user and is held through a userID and gameID
3. The top N players will return a list of game entries sorted solely by highest score (because it's a leaderboard)

Class Diagram Plan:

For classes, we could do something like LeaderboardClient/LeaderboardServer, or other things I haven't thought of.

- LeaderboardClient would be the local instance, and would carry PlayerID and current score.

- LeaderboardServer is the second actor, and would have methods to query certain data

Client side, we can/will incorporate:

1. sendScore(gameID, score)
2. requestTopScores(gameID, max) \\alter to consider other filter orders, OR make it grab and have something else do the filtering, OR make multiple functions to perform different filtering (means filter calls for every call to read values)
3. requestUserTopScore(userID, gameID)
4. currentUserID
5. currentUsername
6. isLoggedIn

Network/Server side, we will incorporate:

1. setScore(userID, gameID, score)
2. getTopScores(gameID, Max)
3. getUserHighTopScore(userID, gameID)
4. handleClient(socket) → **I GOT THIS FROM the platform core team's info - it is a server-wide network handler (from what I interpret it as) that runs when a client connects**

Mapping between use cases and methods (left to right on the table)

| **Use Case** | *Method* | *Methods* |
|---|---|---|
| viewLeaderboard | requestTopScores(gameID, Max) | NetworkServer |
| viewMyScore | requestUserTopScore | Database |
| submitScore | sendScore | Session |

What We Are Creating:

1. A use case diagram for the leaderboard
2. A class diagram for the leaderboard user/server view of everything
3. Will use draw.io to stay consistent with the platform core team